Design and Implementation of an FPGA-Based Obstacle Avoidance System Using Verilog.

23ECE383 VLSI Design Laboratory

B. Tech. ECE

Batch: C1

2025-2026 Odd Semester

CB.EN.U4ECE23207	ASHWIN K
CB.EN.U4ECE23212	CHITTESH S
CB.EN.U4ECE23225	KARTHICK A

Faculty In-charges: **1) Dr. Prabhu E.**

2) Dr. Anita J. P.

Signature of the Faculty:

Department of Electronics and Communication Engineering

Amrita School of Engineering

Amrita Vishwa Vidyapeetham

Amritanagar, Coimbatore – 641112

1. Objective:

The objective of this project is to develop a Verilog-based FSM for ultrasonic distance sensing to provide accurate real-time measurements from 0 cm to 49 cm with an error margin of ±1 cm. The system enhances vehicle safety by incorporating a collision detection and alert mechanism.

- Displaying distance on a 7-segment display for clear visual feedback.
- Transmitting distance wirelessly via an ESP32 module for remote monitoring.
- Creating a local hosted website to display the distance and alert to User via dashboard infotainment system.
- Ensuring robust FPGA implementation, validated through synthesis, resource analysis, and performance evaluation.

2. Software hardware used:

Software:

- Xilinx Vivado Design Suite 2025.1
- ModelSim - Intel FPGA Starter Edition 10.5b (Quartus Prime 18.1)
- Arduino IDE

Hardware:

- Basys 3 FPGA Board
- Digilent Pmod ESP32 Module
- HC-SR04 Ultrasonic sensor
- USB to TTL Convertor (For Pmod ESP32)

3. Overview of the system:

The Ultrasonic Distance Measurement System is designed using a Finite State Machine (FSM) model to ensure precise, real-time, and predictable operation. The system interfaces an ultrasonic sensor with an FPGA to measure distances, display them on a 7-segment display, and transmit the data to an ESP32 module for wireless monitoring. The design also includes a collision detection and alert mechanism to enhance vehicle safety.

System Workflow:

1) **Reset State:** Initializes the system, clears previous measurements, and prepares the sensor and display for a new reading.
2) **Trigger Generation State:** Sends a 10 µs pulse to the ultrasonic sensor to start a distance measurement cycle.
3) **Echo Detection State:** Monitors the rising edge of the echo signal and counts the pulse width using a high-frequency counter. A timeout ensures the system handles cases when no echo is received.
4) **Distance Calculation State:** Converts the measured pulse width into distance in centimetres using a calibrated formula. Values exceeding the sensor range are capped.
5) **BCD Conversion State:** Converts the calculated distance into tens (cm1) and ones (cm0) digits for display purposes.
6) **Display Update State:** Updates the multiplexed 7-segment display with the latest BCD values, ensuring clear and stable visualization.
7) **UART Transmission State:** Detects changes in distance and, if UART is idle, transmits the new distance to the ESP32 for remote monitoring.
8) **Cooldown State:** Waits for 25 ms to ensure sensor stability before initiating the next measurement cycle.

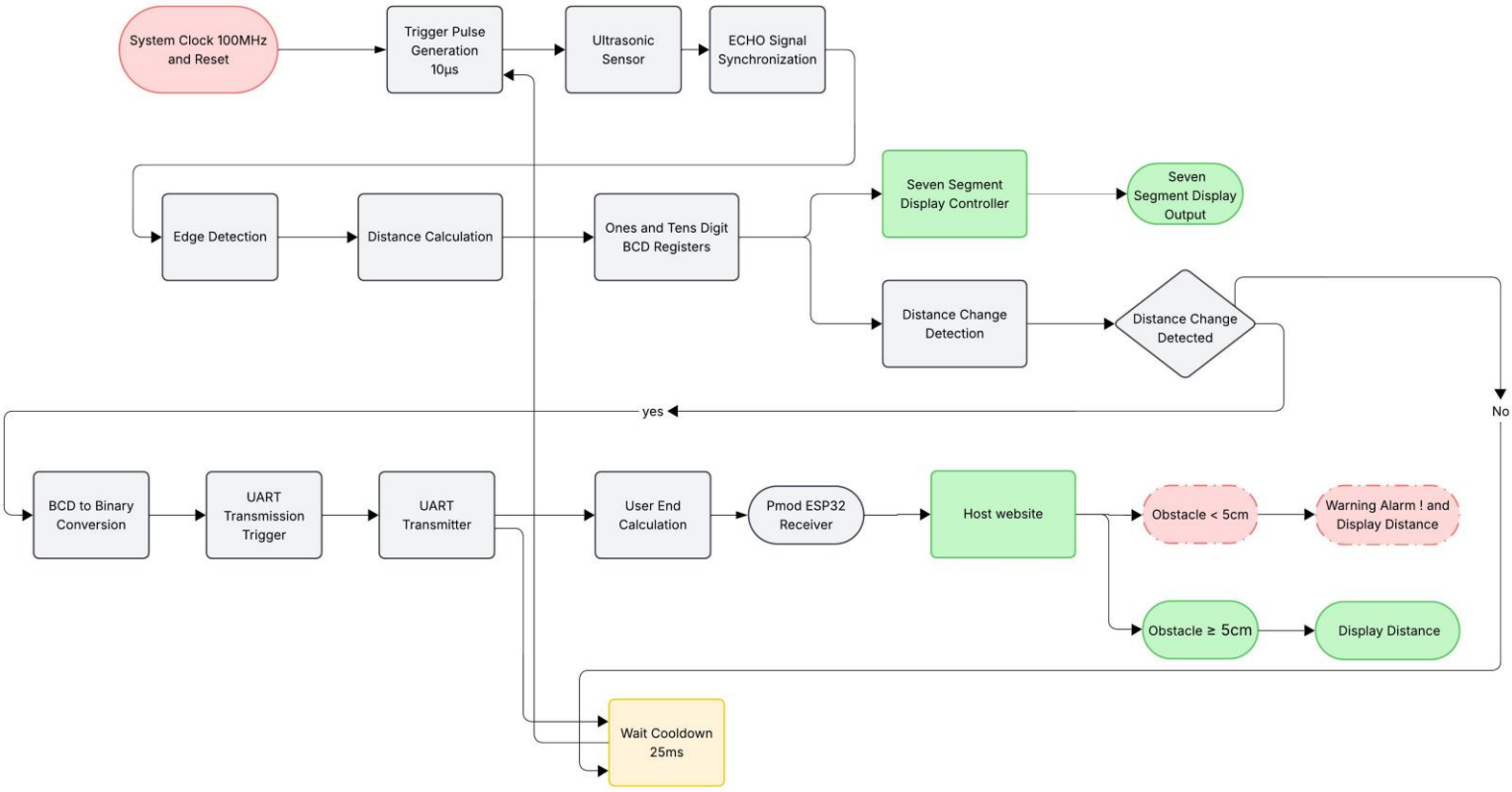
Technical Specifications:

1) Distance Range: 0–49 cm
2) Error probability: ±1 cm
3) Update Rate: 40 Hz (based on measurement + cooldown period)
4) Display: Multiplexed 7-segment (2 digits)
5) UART Baud Rate: 1,15,200

Outputs:

1) Multiplexed 7-segment display showing measured distance (cm).
2) UART transmission of binary distance values to the ESP32.
3) The ESP32 contains the website code with Ip address to host locally. It will display and alert to the user via that website.
4) Optional collision alert signal if distance falls below a safety threshold.

4. Block diagram:



5. Introduction:

In modern vehicles, accurate sensing of the surrounding environment is essential for collision prevention. Traditional proximity sensors often have limited range, slow response, and are sensitive to environmental factors.

FPGA technology provides a high-performance platform for real-time distance measurement using hardware parallelism and digital processing. The Ultrasonic Distance Measurement System emits high-frequency pulses and measures echo return times, enabling: non-contact detection, high accuracy (±1 cm), wide range (0–49 cm). With FSM-based control, 7-segment display, and ESP32 wireless communication, the system offers continuous monitoring, real-time alerts, and remote distance logging, enhancing vehicle safety and collision avoidance.

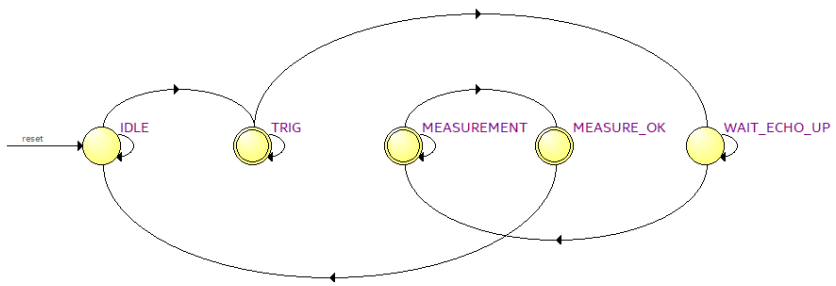


Fig.1 FSM Diagram

6. **Complete Code with comments:**

top.v

```
module project_top (

    // Physical Inputs/Outputs
    input  wire        clk,            // System clock (e.g., 100MHz)
    input  wire        reset,          // Asynchronous reset (active high)

    // Ultrasonic Sensor Interface
    output wire        trig,           // Trigger pulse output to the ultrasonic sensor
    input  wire        echo,           // Echo pulse input from the ultrasonic sensor

    // 7-Segment Display Interface
    output wire [6:0]   seg,           // 7 segment lines (a, b, c, d, e, f, g)
    output wire [3:0]   an,            // Anode select lines for digit multiplexing

    // BCD Debug Outputs (connected to LEDs)
    output wire [3:0]   cm0,           // Ones digit of the distance in BCD (cm)
    output wire [3:0]   cm1,           // Tens digit of the distance in BCD (cm)

    // UART Interface to ESP32
    output wire        uart_tx_pin     // Serial data output pin
);

    //==================================================================
    // Internal Signals for Connecting Modules
    //==================================================================
    wire       uart_busy;             // Indicates if the UART transmitter is currently busy
    wire [7:0]  new_distance;          // Distance value converted to binary (0-255)

    reg        send_trigger;          // Flag to start a UART transmission
    reg  [7:0]  distance_binary;       // Distance value to be loaded into the UART buffer
    reg  [3:0]  prev_cm0;              // Previous value of the ones digit (for change detection)
    reg  [3:0]  prev_cm1;              // Previous value of the tens digit (for change detection)

    //==================================================================
    // BCD to Binary Conversion
    //==================================================================
    // Combines the BCD tens (cm1) and ones (cm0) digits into a single 8-bit binary value.
    assign new_distance = (cm1 * 10) + cm0;

    //==================================================================
    // 1. Instantiate Sonar Module
    //==================================================================
    // This module handles:
```

```
// a) Generating the 'trig' pulse.
// b) Measuring the 'echo' pulse duration to calculate the distance.
// c) Driving the 'seg' and 'an' for the 7-segment display multiplexer.
// d) Providing the calculated distance as BCD outputs 'cm1' and 'cm0'.
sonar sonar_inst (
    .clk      (clk),
    .trig     (trig),
    .echo     (echo),
    .cm1      (cm1),
    .cm0      (cm0),
    .seg      (seg),
    .an       (an)
);

//=====================================================================
// 2. Instantiate UART Transmitter Module
//=====================================================================
// This module handles asynchronous serial communication.
// It transmits the 'i_data' byte when 'i_start' is pulsed high.
uart_tx #(
    // Parameter definitions for clock and baud rate calculation
    .CLOCK_FREQ (100_000_000),      // System clock frequency (Hz)
    .BAUD_RATE  (115200)            // Desired baud rate (bps)
) uart_inst (
    .clk      (clk),            // System clock
    .i_rst    (reset),          // Reset signal
    .i_start  (send_trigger),   // Start transmission trigger
    .i_data   (distance_binary), // 8-bit data to transmit
    .o_tx     (uart_tx_pin),    // Serial transmit line output
    .o_busy   (uart_busy)       // UART busy status output
);

//=====================================================================
// 3. Glue Logic (Manages UART Transmission)
//=====================================================================
// This always block manages the state of the UART transmitter,
// ensuring data is only sent when the distance changes and the UART is free.
always @(posedge clk) begin
    // Store the current BCD values to detect a change in the next cycle.
    prev_cm0 <= cm0;
    prev_cm1 <= cm1;

    // Default the trigger to low (self-clearing pulse).
    send_trigger <= 1'b0;

    // Check if:
    // 1. The distance has changed (cm0 or cm1 is different from its previous value)
    // AND
```

```
        // 2. The UART is not busy.
        if ((cm0 != prev_cm0 || cm1 != prev_cm1) && !uart_busy) begin
            // A change was detected!
            // Load the new binary distance value into the UART input register.
            distance_binary <= new_distance;
            // Pulse the 'send_trigger' high for one clock cycle to start transmission.
            send_trigger <= 1'b1;
        end
    end

endmodule
```

sonar.v

```
module sonar (
    input  wire clk,              // System clock (e.g., 50MHz or 100MHz)
    output reg  trig,             // Output: 10us Trigger pulse to the HC-SR04 sensor
    input  wire echo,             // Input: Echo pulse from the sensor (duration is proportional to
distance)
    output reg [3:0] cm1,         // Output: Tens digit of the distance in BCD (cm)
    output reg [3:0] cm0,         // Output: Ones digit of the distance in BCD (cm)
    output reg [6:0] seg,         // Output: 7-segment display segments (a-g, active low)
    output reg [3:0] an           // Output: 7-segment display digit select (anodes, active low)
);

    //==================================================================
    // Internal Signals & Registers
    //==================================================================

    // Clock divider for display refresh
    // [16:0] counter provides a slow signal for display multiplexing.
    reg [16:0] clk_div = 17'd0;

    // Display signals
    reg [3:0] display_digit = 4'd0;    // The BCD value of the digit currently being displayed
    reg [1:0] digit_sel = 2'd0;        // Selects which of the two digits is active (00=ones,
01=tens)

    // Distance registers (Intermediate BCD values used during counting)
    reg [3:0] dist_ones = 4'd0;        // Accumulates the ones digit during the measurement phase
    reg [3:0] dist_tens = 4'd0;        // Accumulates the tens digit during the measurement phase

    // Registered outputs (Buffers the calculated and display values)
    reg [3:0] cm0_reg = 4'd0;
    reg [3:0] cm1_reg = 4'd0;
```

```
   reg [6:0] seg_reg = 7'b1111111;    // Initialize to 'off' (all segments high for active low
display)
   reg [3:0] an_reg  = 4'b1111;       // Initialize to 'off' (all anodes high for active low display)

   // Echo synchronization (to prevent metastability)
   reg [2:0] echo_sync = 3'b000;      // 3-stage shift register for input synchronization
   reg echo_synced = 1'b0;            // Stable, synchronized version of 'echo'
   reg prev_echo_synced = 1'b0;       // Previous cycle's synchronized 'echo' for edge detection

   // Edge detection registers
   reg echo_rising = 1'b0;            // Pulsed high on the rising edge of the echo
   reg echo_falling = 1'b0;           // Pulsed high on the falling edge of the echo

   //==================================================================
   // Constants for Timing and Measurement
   //==================================================================
   // Assuming a 50MHz clock (clk period = 20ns). Adjust if clock is different.
   localparam TRIG_DURATION        = 500;       // 10 us trigger pulse (10us / 20ns = 500
cycles)
   // Formula for COUNTS_PER_CM: (2 * SpeedOfSound * clock_freq) / 100
   // SpeedOfSound = 343 m/s = 34.3 cm/ms.
   // Time for 1cm (round trip) = (2 * 1 cm) / 34300 cm/s = 58.3 us
   // Cycles for 1cm @ 50MHz = 58.3 us / 20 ns = 2915 cycles. Using 2900 for
simplicity/calibration.
   localparam COUNTS_PER_CM        = 2900;      // Clock cycles equivalent to 1 cm of
distance
   localparam TIMEOUT_COUNT        = 700000;    // Timeout ~14ms (max measurement
range of ~240 cm)
   localparam MEASUREMENT_INTERVAL = 1250000;   // ~25ms minimum time between
measurements

   // State machine encoding
   localparam IDLE             = 3'd0,       // Waiting for the next measurement cycle
              TRIGGER            = 3'd1,       // Generating the 10us trigger pulse
              WAIT_ECHO_START     = 3'd2,       // Waiting for the rising edge of the echo
pulse
              WAIT_ECHO_END       = 3'd3,       // Counting the echo pulse duration
              UPDATE             = 3'd4,       // Latching the new distance to the output registers
              COOLDOWN            = 3'd5;       // Waiting for the minimum measurement
interval

   reg [2:0] state = IDLE;                   // Current state of the sonar FSM

   // Counters for timing and distance calculation
   integer distance_counter = 0;              // Counts cycles within 1 cm during
WAIT_ECHO_END
   integer measurement_counter = 0;           // General-purpose counter for timing states
(TRIG, WAIT, COOLDOWN)
```

```
    reg reading_valid = 1'b0;                // Flag to indicate a successful distance reading
    integer reset_counter = 0;               // Used to clear the display after a timeout/invalid
reading

    //==================================================================
    // Combinational Output Assignments
    //==================================================================
    // Assign the buffered registers to the module's outputs.
    always @(*) begin
        cm0 = cm0_reg;
        cm1 = cm1_reg;
        seg = seg_reg;
        an  = an_reg;
    end

    //==================================================================
    // Main Sonar FSM and Measurement Process
    //==================================================================
    always @(posedge clk) begin
        // --- 1. Echo Synchronization ---
        // Shift the raw 'echo' input through the synchronization pipeline.
        echo_sync <= {echo_sync[1:0], echo};
        prev_echo_synced <= echo_synced;
        echo_synced <= echo_sync[2];

        // --- 2. Edge Detection ---
        // Logic to detect transitions on the stable 'echo_synced' signal.
        echo_rising  <= (echo_synced & ~prev_echo_synced);
        echo_falling <= (~echo_synced & prev_echo_synced);

        // --- 3. State Machine Logic ---
        case (state)
            IDLE: begin
                // Reset all measurement variables and move to the TRIGGER state.
                trig <= 1'b0;
                distance_counter <= 0;
                reading_valid <= 1'b0;
                measurement_counter <= 0;
                state <= TRIGGER;
            end

            TRIGGER: begin
                // Generate the required 10us pulse on 'trig'.
                if (measurement_counter < TRIG_DURATION) begin
                    trig <= 1'b1;
                    measurement_counter <= measurement_counter + 1;
                end else begin
                    // Trigger pulse complete.
```

```
            trig <= 1'b0;
            measurement_counter <= 0;
            state <= WAIT_ECHO_START;
        end
    end

    WAIT_ECHO_START: begin
        // Wait for the rising edge of the echo pulse.
        if (echo_rising) begin
            // Echo received, start measuring.
            distance_counter <= 0;
            dist_ones <= 4'b0000; // Reset BCD accumulation
            dist_tens <= 4'b0000;
            state <= WAIT_ECHO_END;
            measurement_counter <= 0;
        end else if (measurement_counter > TIMEOUT_COUNT) begin
            // Timeout occurred before the echo rising edge. Assume no object or out of
range.
            cm0_reg <= 4'b0000; // Display 00
            cm1_reg <= 4'b0000;
            reading_valid <= 1'b0;
            state <= COOLDOWN;
        end else begin
            // Keep counting for timeout.
            measurement_counter <= measurement_counter + 1;
        end
    end

    WAIT_ECHO_END: begin
        // Measure the duration of the high echo pulse.
        if (echo_falling) begin
            // Measurement complete. Latch result.
            reading_valid <= 1'b1;
            state <= UPDATE;
        end else if (measurement_counter > TIMEOUT_COUNT) begin
            // Timeout occurred before the echo falling edge.
            reading_valid <= 1'b0;
            state <= COOLDOWN;
        end else begin
            // Increment measurement counter (for timeout) and distance counter (for cm
calculation).
            measurement_counter <= measurement_counter + 1;

            // Distance-to-BCD Conversion Logic (runs every clock cycle)
            if (distance_counter >= COUNTS_PER_CM) begin
                // One centimeter of distance has elapsed. Increment BCD registers.
                distance_counter <= 0; // Reset cycles for the next centimeter
                if (dist_ones == 4'd9) begin
```

```
                // Ones overflow, carry to tens.
                if (dist_tens == 4'd9) begin
                    // Maximum range (99 cm) reached. Stop counting at 99.
                    dist_ones <= 4'd9;
                    dist_tens <= 4'd9;
                end else begin
                    // Increment tens digit.
                    dist_ones <= 4'd0;
                    dist_tens <= dist_tens + 1;
                end
            end else begin
                // Simple increment of the ones digit.
                dist_ones <= dist_ones + 1;
            end
        end else begin
            // Not yet 1 cm, keep counting cycles.
            distance_counter <= distance_counter + 1;
        end
    end
end

UPDATE: begin
    // Latch the final calculated BCD distance to the output registers.
    if (reading_valid) begin
        cm0_reg <= dist_ones;
        cm1_reg <= dist_tens;
    end
    measurement_counter <= 0;
    state <= COOLDOWN;
end

COOLDOWN: begin
    // Wait for the required MEASUREMENT_INTERVAL before starting a new
cycle.
    if (measurement_counter < MEASUREMENT_INTERVAL) begin
        measurement_counter <= measurement_counter + 1;
    end else begin
        // Cooldown complete. Prepare for IDLE.
        dist_ones <= 4'b0000; // Reset BCD accumulators for the next cycle
        dist_tens <= 4'b0000;

        // Small counter to occasionally reset the display if no valid reading.
        if (reset_counter < 10)
            reset_counter <= reset_counter + 1;
        else begin
            reset_counter <= 0;
            if (!reading_valid) begin
                // Clear display outputs if the last reading was a timeout.
```

```
                cm0_reg <= 4'b0000;
                cm1_reg <= 4'b0000;
              end
            end
            state <= IDLE; // Start the next measurement
          end
        end
      endcase
    end

// Declared outside the main block for cleaner organization
reg [6:0] seg_pattern;
reg [3:0] an_pattern;

    //=====================================================================
    // 7-Segment Display Controller (Multiplexing)
    //=====================================================================
    // This process uses 'clk_div' to multiplex the two digits (cm1, cm0) onto
    // the single 7-segment bus for visual display.
    always @(posedge clk) begin
      // --- 1. Clock Divider and Digit Selection ---
      clk_div <= clk_div + 1;
      // Use the upper bits of the divider for a slower selection rate (~12 Hz refresh)
      digit_sel <= clk_div[16:15];

      // --- 2. Multiplexer Logic ---
      case (digit_sel)
        2'b00: begin
          // Display the Ones digit (Rightmost digit)
          display_digit <= cm0_reg;
          an_pattern = 4'b1110; // Activate Anode 0
        end
        2'b01: begin
          // Display the Tens digit (Leftmost digit)
          display_digit <= cm1_reg;
          an_pattern = 4'b1101; // Activate Anode 1
        end
        2'b10, 2'b11: begin
          // Blank the display momentarily to prevent ghosting (Optional blanking frames)
          display_digit <= 4'b0000;
          an_pattern = 4'b1111; // All anodes off
        end
      endcase

      // --- 3. BCD to 7-Segment Decoder ---
      // Converts the BCD value of the selected digit into the segment pattern (Active Low).
      case (display_digit)
        4'd0: seg_pattern = 7'b1000000; // 0
```

```
            4'd1: seg_pattern = 7'b1111001; // 1
            4'd2: seg_pattern = 7'b0100100; // 2
            4'd3: seg_pattern = 7'b0110000; // 3
            4'd4: seg_pattern = 7'b0011001; // 4
            4'd5: seg_pattern = 7'b0010010; // 5
            4'd6: seg_pattern = 7'b0000010; // 6
            4'd7: seg_pattern = 7'b1111000; // 7
            4'd8: seg_pattern = 7'b0000000; // 8
            4'd9: seg_pattern = 7'b0010000; // 9
            default: seg_pattern = 7'b0111111; // Error/Blank (E)
        endcase

        // --- 4. Latch Display Outputs ---
        seg_reg <= seg_pattern;
        an_reg  <= an_pattern;
    end

endmodule
```

uart.v

```
module uart_tx #(
    // --- Configuration Parameters ---
    parameter CLOCK_FREQ = 100_000_000, // System clock frequency (Hz)
    parameter BAUD_RATE  = 115200       // Desired serial transmission rate (bits per second)
)(
    // --- Port Declarations ---
    input  wire clk,                // System clock input
    input  wire i_rst,              // Asynchronous reset (active high)
    input  wire i_start,            // Pulsed high to initiate a transmission
    input  wire [7:0] i_data,       // 8-bit data byte to be transmitted
    output wire o_tx,               // Serial data output pin (TX)
    output wire o_busy              // Status flag: High when a transmission is in progress
);
    //==================================================================
    // Timing and State Constants
    //==================================================================
    // Calculate the number of clock cycles required for one bit time.
    localparam CLKS_PER_BIT = CLOCK_FREQ / BAUD_RATE;
    // Standard UART data length.
    localparam DATA_BITS = 8;

    // FSM State Encodings
    localparam IDLE = 2'b00,          // Waiting for a transmit request (i_start)
               START_BIT = 2'b01,     // Transmitting the mandatory Start bit (LOW)
```

```
        DATA_BITS_STATE = 2'b10, // Transmitting the 8 data bits (LSB first)
        STOP_BIT = 2'b11;        // Transmitting the mandatory Stop bit (HIGH)

//==================================================================
// Internal Registers
//==================================================================
reg [1:0] r_state = IDLE;                  // Current state register for the FSM
reg [7:0] r_tx_data;                       // Register to hold the data byte being transmitted
// Clock counter register (size depends on CLKS_PER_BIT)
reg [$clog2(CLKS_PER_BIT)-1:0] r_clk_count = 0;
reg [3:0] r_bit_index = 0;                 // Tracks the index of the data bit being sent (0 to 7)
reg r_tx_pin = 1'b1;                       // The output signal register. Held HIGH in IDLE.

//==================================================================
// FSM and Transmission Logic
//==================================================================
always @(posedge clk) begin
   if (i_rst) begin
      // --- Asynchronous Reset ---
      r_state <= IDLE;
      r_clk_count <= 0;
      r_bit_index <= 0;
      r_tx_pin <= 1'b1; // Ensure TX pin is high (IDLE state)
   end else begin
      // --- State Transition Logic ---
      case (r_state)
         IDLE: begin
            r_clk_count <= 0;
            r_bit_index <= 0;
            if (i_start) begin
               // Transmission requested: Latch data and begin Start bit.
               r_tx_data <= i_data; // Latch input data
               r_state <= START_BIT; // Transition to start bit
               r_tx_pin <= 1'b0; // Start bit is mandatory LOW
            end else begin
               r_tx_pin <= 1'b1; // Idle state must be HIGH
            end
         end

         START_BIT: begin
            if (r_clk_count < CLKS_PER_BIT - 1) begin
               // Hold the Start bit for a full bit duration.
               r_clk_count <= r_clk_count + 1;
            end else begin
               // Start bit done; transition to data bits.
               r_clk_count <= 0;
               r_state <= DATA_BITS_STATE;
               // Set the pin to the first data bit (D0).
```

```
                r_tx_pin <= r_tx_data[0]; // **Correction: Changed index to D0 (LSB) to start
transmission.**
                r_bit_index <= 1;          // Set index for the next bit (D1)
            end
        end

        DATA_BITS_STATE: begin
            if (r_clk_count < CLKS_PER_BIT - 1) begin
                // Hold the current data bit for a full bit duration.
                r_clk_count <= r_clk_count + 1;
            end else begin
                // Bit duration elapsed; transition to the next bit or Stop bit.
                r_clk_count <= 0;
                if (r_bit_index < DATA_BITS) begin
                    // Transmit the next data bit (D1 through D7).
                    r_tx_pin <= r_tx_data[r_bit_index]; // Update to next data bit
                    r_bit_index <= r_bit_index + 1; // Increment bit index
                end else begin
                    // All 8 data bits sent; transition to Stop bit.
                    r_state <= STOP_BIT;
                    r_tx_pin <= 1'b1; // Stop bit is mandatory HIGH
                end
            end
        end

        STOP_BIT: begin
            if (r_clk_count < CLKS_PER_BIT - 1) begin
                // Hold the Stop bit for a full bit duration.
                r_clk_count <= r_clk_count + 1;
            end else begin
                // Stop bit done; return to idle.
                r_clk_count <= 0;
                r_state <= IDLE;
                r_tx_pin <= 1'b1; // Ensure pin is high in idle
            end
        end

        default: begin
            // Safety: Catch undefined states.
            r_state <= IDLE;
        end
    endcase
  end
end

//=================================================================
// Output Assignments
//=================================================================
```

```
    assign o_tx = r_tx_pin;              // Connect the output register to the physical pin
    assign o_busy = (r_state != IDLE);      // Busy signal is asserted whenever the FSM is not in
IDLE
endmodule
```

Constarint.xdc

```
# Basys 3 Master Clock
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} [get_ports clk]

## Reset Button
set_property -dict { PACKAGE_PIN U18  IOSTANDARD LVCMOS33 } [get_ports reset];

## UART Transmit Signal (output to ESP32)
set_property -dict { PACKAGE_PIN A16  IOSTANDARD LVCMOS33 } [get_ports {uart_tx_pin}];

## Trigger signal (output to ultrasonic sensor)
set_property -dict { PACKAGE_PIN K17  IOSTANDARD LVCMOS33 } [get_ports trig];

## Echo signal (input from ultrasonic sensor)
set_property -dict { PACKAGE_PIN N17  IOSTANDARD LVCMOS33 } [get_ports echo];

## 7-Segment Display (segments)
set_property -dict { PACKAGE_PIN W7   IOSTANDARD LVCMOS33 } [get_ports {seg[0]}];
set_property -dict { PACKAGE_PIN W6   IOSTANDARD LVCMOS33 } [get_ports {seg[1]}];
set_property -dict { PACKAGE_PIN U8   IOSTANDARD LVCMOS33 } [get_ports {seg[2]}];
set_property -dict { PACKAGE_PIN V8   IOSTANDARD LVCMOS33 } [get_ports {seg[3]}];
set_property -dict { PACKAGE_PIN U5   IOSTANDARD LVCMOS33 } [get_ports {seg[4]}];
set_property -dict { PACKAGE_PIN V5   IOSTANDARD LVCMOS33 } [get_ports {seg[5]}];
set_property -dict { PACKAGE_PIN U7   IOSTANDARD LVCMOS33 } [get_ports {seg[6]}];

## 7-Segment Display (anodes)
set_property -dict { PACKAGE_PIN U2   IOSTANDARD LVCMOS33 } [get_ports {an[0]}];
set_property -dict { PACKAGE_PIN U4   IOSTANDARD LVCMOS33 } [get_ports {an[1]}];
set_property -dict { PACKAGE_PIN V4   IOSTANDARD LVCMOS33 } [get_ports {an[2]}];
set_property -dict { PACKAGE_PIN W4   IOSTANDARD LVCMOS33 } [get_ports {an[3]}];

## Basys 3 LEDs (for debugging BCD outputs)
set_property -dict { PACKAGE_PIN U16  IOSTANDARD LVCMOS33 } [get_ports {cm0[0]}];
set_property -dict { PACKAGE_PIN E19  IOSTANDARD LVCMOS33 } [get_ports {cm0[1]}];
set_property -dict { PACKAGE_PIN U19  IOSTANDARD LVCMOS33 } [get_ports {cm0[2]}];
set_property -dict { PACKAGE_PIN V19  IOSTANDARD LVCMOS33 } [get_ports {cm0[3]}];

set_property -dict { PACKAGE_PIN W18  IOSTANDARD LVCMOS33 } [get_ports {cm1[0]}];
set_property -dict { PACKAGE_PIN U15  IOSTANDARD LVCMOS33 } [get_ports {cm1[1]}];
set_property -dict { PACKAGE_PIN U14  IOSTANDARD LVCMOS33 } [get_ports {cm1[2]}];
set_property -dict { PACKAGE_PIN V14  IOSTANDARD LVCMOS33 } [get_ports {cm1[3]}];
```


*Important Note:
This project has Totally 3 Modules. In that top.v, sonar.v, uart.v are placed according to the Hierarchy. Constraint file has all required pin mapping. Arduino IDE is used to program ESP32 (C++ language is Used).

C++ Code For ESP32:

```
// These lines include the necessary libraries for the project.
#include <WiFi.h>        // Manages WiFi connection for the ESP32.
#include <WebServer.h>   // Allows the ESP32 to act as a web server.
#include <HardwareSerial.h> // Manages hardware-based serial communication (UART).

// IMPORTANT: Replace these with the actual credentials for your WiFi network.
const char* ssid = "test";      // The name (SSID) of your WiFi network.
const char* password = "12345678"; // The password for your WiFi network.

// Defines the settings for serial communication with the FPGA.
const int uartRxPin = 16;     // The ESP32 pin used to receive data from the FPGA (RX).
const int uartTxPin = -1;     // The transmit pin (TX). Set to -1 because we are only receiving data.
const long baudRate = 115200; // The communication speed in bits per second. Must match the
FPGA's setting.

// These objects are created once and are accessible from anywhere in the code.
HardwareSerial uartSerial(2); // Creates a serial object using UART port 2 of the ESP32.
WebServer server(80);         // Creates a web server object that listens on port 80 (the standard HTTP
port).
volatile int fpgaValue = -1;  // A variable to store the value read from the FPGA.
                              // 'volatile' tells the compiler that this value can change unexpectedly,
                              // which is important as it's modified in the main loop and accessed by the web
server.
                              // Initialized to -1 to indicate no data has been received yet.

// This function runs every time a user's web browser requests the root URL ("/") of the ESP32's IP
address.
// Its job is to generate and send the HTML page to the browser.
void handleRoot() {
  // Create a String variable to build the HTML content.
  String html = "<!DOCTYPE html><html><head><title>FPGA Sensor Data</title>";

  // This meta tag tells the browser to automatically refresh the page every 0.25 seconds.
```

```
  // This allows the data to appear "live" without manual refreshing.
  html += "<meta http-equiv='refresh' content='0.25'>";

  // This section contains the CSS code to style the webpage.
  // CSS controls the appearance (colors, fonts, layout) of HTML elements.
  html += "<style>";
  html += "body{font-family:Arial,sans-serif;text-align:center;margin-top:50px;font-
size:1.5em;background-color:#282c34;color:white;}";
  html += "h1{color:#61dafb;}";
  // CSS rules for the circular light indicator.
  html += ".light-indicator{width:100px;height:100px;border-radius:50%;margin:20px
auto;border:3px solid #555;}";
  html += ".on{background-color:red;box-shadow:0 0 25px red;}"; // Style for the RED "on" state.
  html += ".off{background-color:#333;}"; // Style for the gray "off" state.
  html += "</style>";

  // This is the main body of the webpage.
  html += "</head><body><h1>Live Value from FPGA</h1>";

  // A paragraph element to display the sensor reading.
  html += "<p style='font-size:4em;font-weight:bold;color:#61dafb;'>";

  // Check if we have received any valid data from the FPGA yet.
  if (fpgaValue < 0) {
    // If not, display a waiting message.
    html += "Waiting for data...";
  } else {
    // If data is available, calculate the distance (value / 2) and display it.
    html += String(fpgaValue / 2) + " cm";
  }

  html += "</p>";

  // This section adds the proximity alert light to the page.
```

```
  html += "<h2>Proximity Alert</h2>";

  // Check if the calculated distance is less than 10 cm.
  // We also check 'fpgaValue >= 0' to ensure we have valid data before checking the distance.
  if (fpgaValue >= 0 && (fpgaValue / 2) < 10) {
    // If the condition is true, create a div with the "on" class, which makes it red.
    html += "<div class='light-indicator on'></div>";
  } else {
    // Otherwise, create a div with the "off" class, which makes it gray.
    html += "<div class='light-indicator off'></div>";
  }

  // Close the HTML tags.
  html += "</body></html>";

  // Send the complete HTML page to the client's web browser with an "OK" status code (200).
  server.send(200, "text/html", html);
}

// The setup() function runs only once when the ESP32 is powered on or reset.
// It is used for initialization tasks.
void setup() {
  // Start serial communication with the computer for debugging purposes (via USB cable).
  Serial.begin(115200);

  // Start serial communication with the FPGA using the defined pins and baud rate.
  uartSerial.begin(baudRate, SERIAL_8N1, uartRxPin, uartTxPin);

  // Begin connecting to the WiFi network.
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi...");

  // This loop waits until the WiFi connection is established.
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500); // Wait for half a second.
    Serial.print(".");
  }

  // Once connected, print the success message and the ESP32's IP address.
  Serial.println("\nWiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  // Tell the web server which function to call (handleRoot) when the root URL is requested.
  server.on("/", handleRoot);

  // Start the web server.
  server.begin();
  Serial.println("HTTP server started.");
}

// The loop() function runs continuously over and over again after setup() is complete.
void loop() {
  // Check if there is any data available to be read from the FPGA via UART.
  if (uartSerial.available() > 0) {
    // If data is available, read the byte and store it in our global variable.
    fpgaValue = uartSerial.read();

    // Print the received value to the serial monitor for debugging.
    Serial.print("Received from FPGA (in CM): ");
    Serial.println(fpgaValue / 2);
  }

  // This is essential. It allows the web server to check for and handle
  // any incoming requests from web clients. This must be called in the loop.
  server.handleClient();
```

7. RTL:

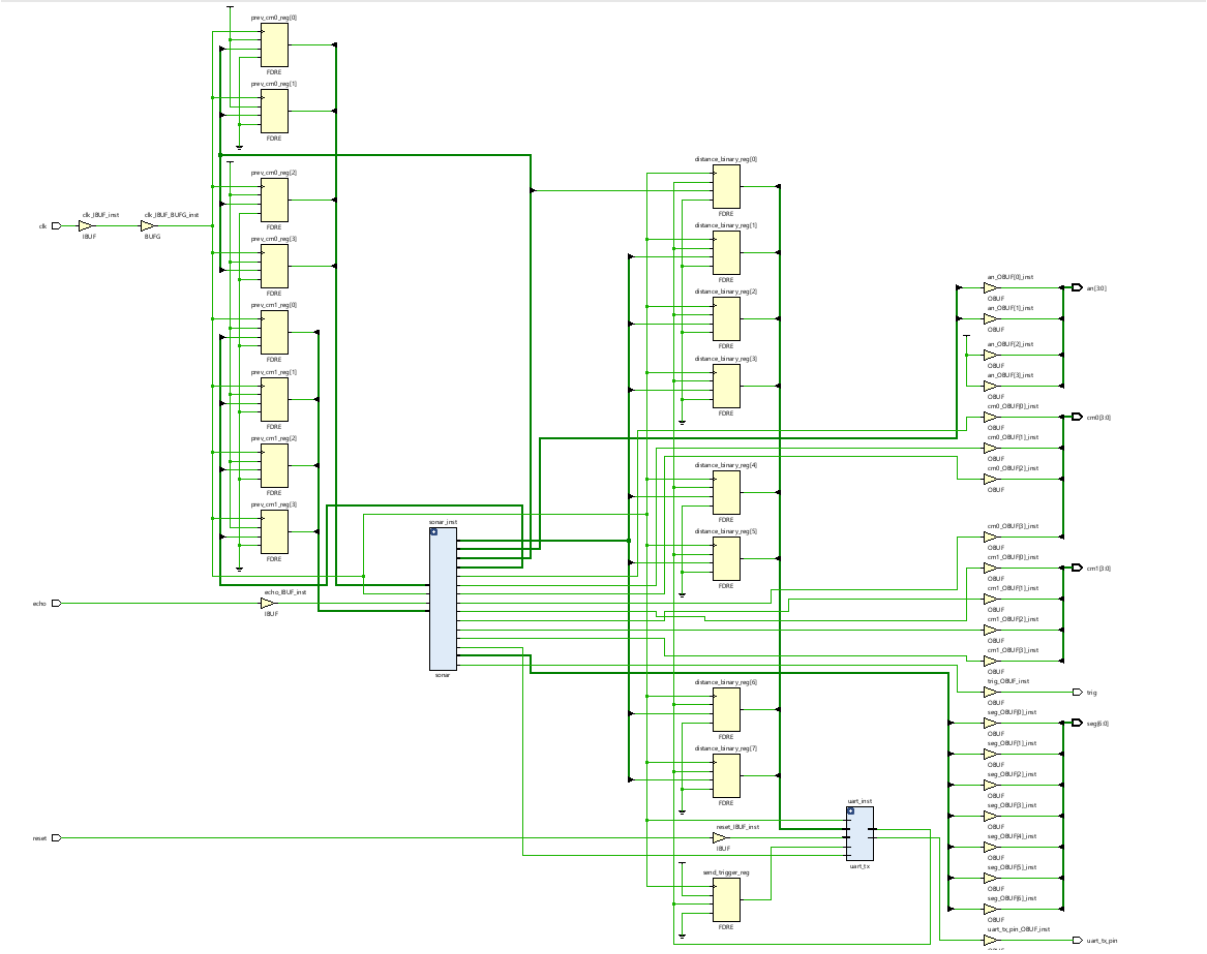


Fig.2 RTL Diagram

8. Resource Utilization summary table:

Resource	Utilization	Available	Utilization %
LUT	234	20800	1.13
LUTRAM	1	9600	0.01
FF	203	41600	0.49
IO	24	106	22.64

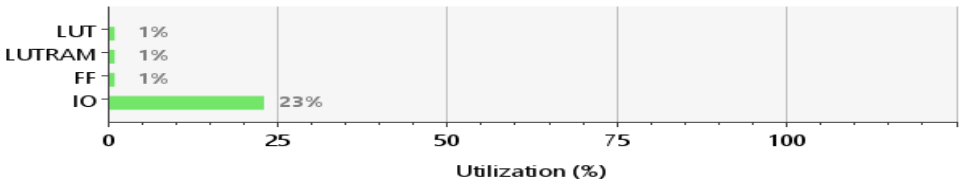


Fig.3 Utilization Report

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.086 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25.4°C**
Thermal Margin:	59.6°C (11.8 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power

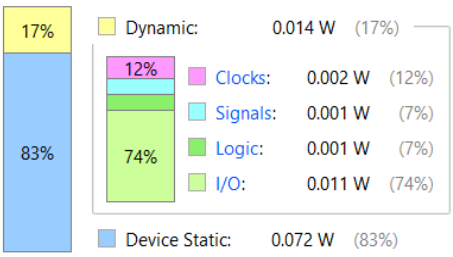


Fig.4 Power Utilization Report

Name	Direction	Package Pin	Fixed	Bank	I/O Std	Vcco	Drive Strength
All ports (24)							
an (4)	OUT		✓	34	LVCMOS33*	3.300	12
an[3]	OUT	W4	✓	34	LVCMOS33*	3.300	12
an[2]	OUT	V4	✓	34	LVCMOS33*	3.300	12
an[1]	OUT	U4	✓	34	LVCMOS33*	3.300	12
an[0]	OUT	U2	✓	34	LVCMOS33*	3.300	12
cm0 (4)	OUT		✓	14	LVCMOS33*	3.300	12
cm0[3]	OUT	V19	✓	14	LVCMOS33*	3.300	12
cm0[2]	OUT	U19	✓	14	LVCMOS33*	3.300	12
cm0[1]	OUT	E19	✓	14	LVCMOS33*	3.300	12
cm0[0]	OUT	U16	✓	14	LVCMOS33*	3.300	12
cm1 (4)	OUT		✓	14	LVCMOS33*	3.300	12
cm1[3]	OUT	V14	✓	14	LVCMOS33*	3.300	12
cm1[2]	OUT	U14	✓	14	LVCMOS33*	3.300	12
cm1[1]	OUT	U15	✓	14	LVCMOS33*	3.300	12
cm1[0]	OUT	W18	✓	14	LVCMOS33*	3.300	12
seg (7)	OUT		✓	34	LVCMOS33*	3.300	12
seg[6]	OUT	U7	✓	34	LVCMOS33*	3.300	12
seg[5]	OUT	V5	✓	34	LVCMOS33*	3.300	12
seg[4]	OUT	U5	✓	34	LVCMOS33*	3.300	12
seg[3]	OUT	V8	✓	34	LVCMOS33*	3.300	12
seg[2]	OUT	U8	✓	34	LVCMOS33*	3.300	12
seg[1]	OUT	W6	✓	34	LVCMOS33*	3.300	12
seg[0]	OUT	W7	✓	34	LVCMOS33*	3.300	12
Scalar ports (5)							
clk	IN	W5	✓	34	LVCMOS33*	3.300	
echo	IN	N17	✓	14	LVCMOS33*	3.300	
reset	IN	U18	✓	14	LVCMOS33*	3.300	
trig	OUT	K17	✓	14	LVCMOS33*	3.300	12
uart_tx_pin	OUT	A16	✓	16	LVCMOS33*	3.300	12

Fig.5 I/O Bank Details

9. Output Waveform and Implemented Hardware Output

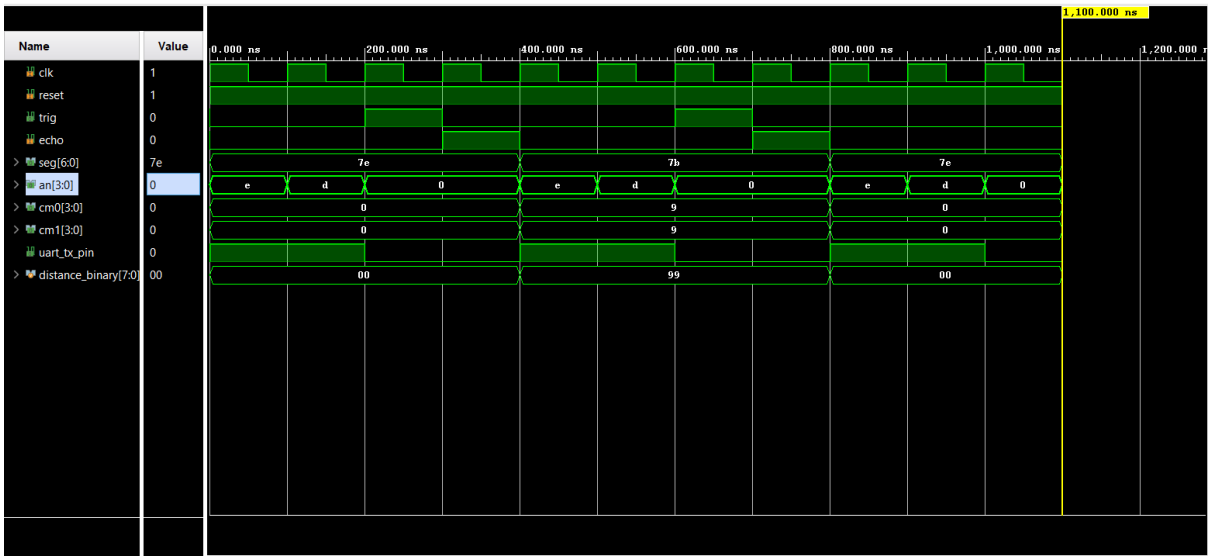


Fig.6.1 Case 1 obstacle ≥ 5cm Simulated Output.

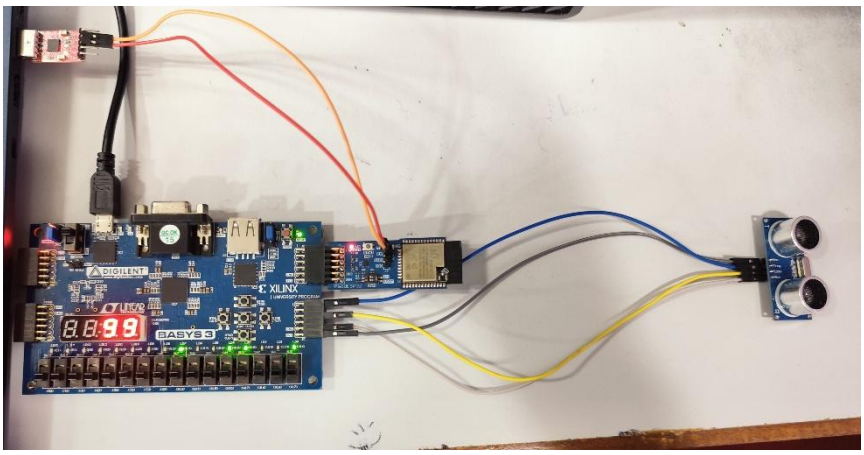


Fig.6.2 Case 1 obstacle ≥ 5cm Implemented Output

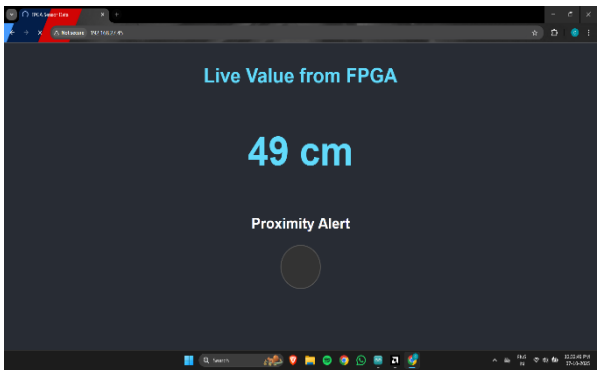


Fig.6.3 Case 1 obstacle ≥ 5cm User Output (99/2 = 49.5 ≅ 49)

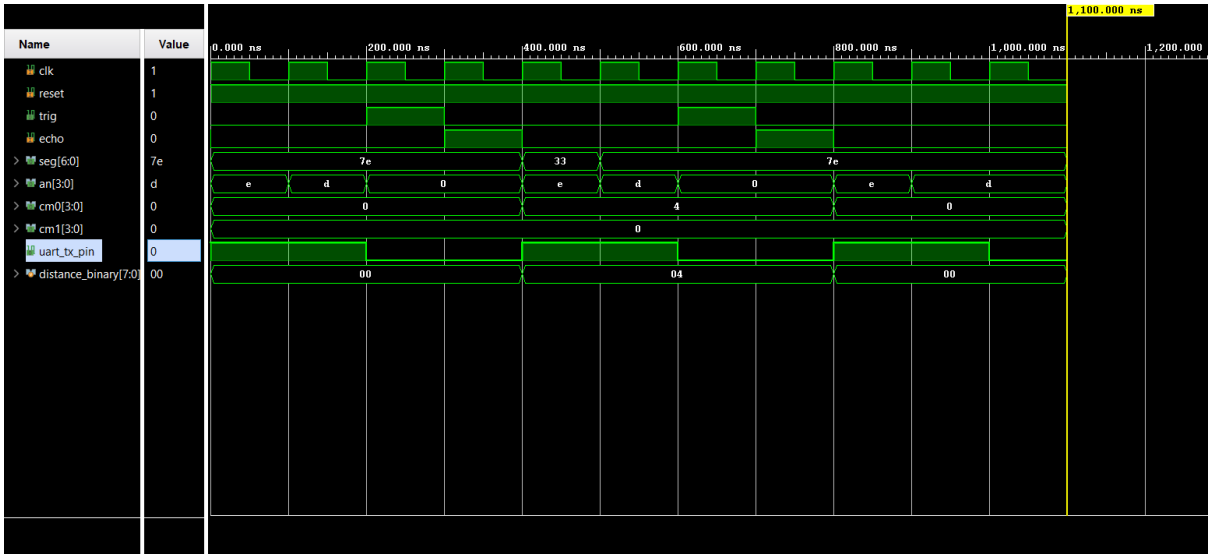


Fig.7.1 Case 2 obstacle < 5cm Simulated Output.

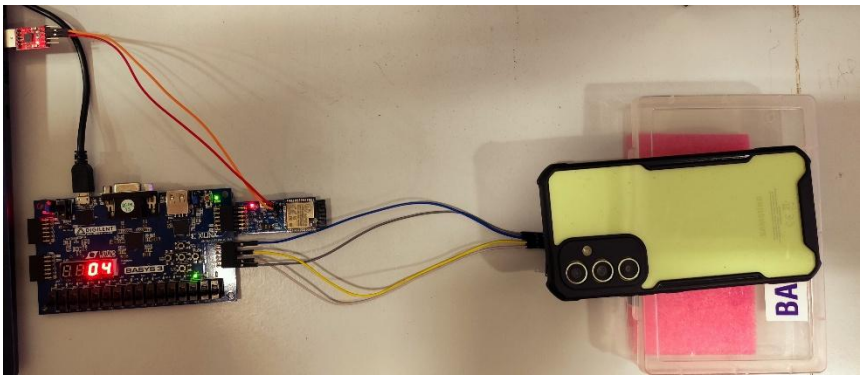


Fig.7.2 Case 2 obstacle < 5cm Implemented Output

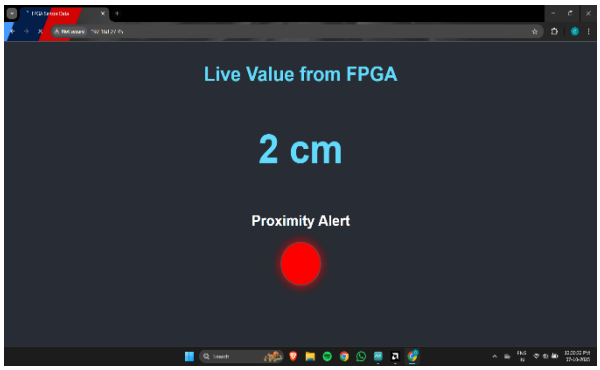


Fig.7.3 Case 2 obstacle < 5cm User Output ALERT ! (4/2 = 2)

*Important Note:
Ultrasonic Sensor Will send the trigger wave and receive the wave and then calculate the distance. So, that it will always show double times as the original distance. In Basys 3 FPGA board the ultrasonic sensor measured distance is displayed and in user side half of the value (the original Distance) is displayed.